

What has been built

A plain account of the content system: **what you asked for, what exists today, what it does, and what is not finished.** Written to be read start to finish in ten minutes.

17 clauses built and audited	116 numbered requirements	95 automated tests passing	0/12 acceptance tests met yet
------------------------------------	---------------------------------	----------------------------------	-------------------------------------

The short version

Every clause in your specification is built and running. You can use the system today: send a voice note, answer a question, approve a post. **Nothing in it is currently known to be broken.**

And the honest half

Nothing has been *accepted* yet. No draft has cleared the quality gate, LinkedIn is not connected, and seven library sections are waiting on you. None of the twelve acceptance tests has been passed — and none has been failed either.

Contents · 1 What you asked for, clause by clause · 2 The three surfaces · 3 What each one does · 4 What else was built · 5 How a post gets made · 6 The rules it cannot break · 7 What is not done · 8 What we need from you · 9 What comes next

Your specification has seventeen clauses and 116 numbered requirements. This is every one of them, what it asked for, and what exists. **Built** means the code is written, deployed and running. **Proven** means it has also been driven against the live system and watched working.

#	WHAT YOU ASKED FOR	WHAT EXISTS	STATE
1	Posts only you could have written	The lived-experience test lives in a locked section of your library. The learning loop cannot propose a change to it — a database rule refuses.	PROVEN
2	Three non-negotiable rules	All three enforced by the database rather than by code: only-you, nothing invented, nothing publishes without you.	PROVEN
3	What is in scope, and what is not	LinkedIn only, one person, text and rebuilt diagrams. No other network, no ghostwriting of anyone else.	PROVEN
4	Five ways material gets in	Voice notes, prompted sessions, call transcripts, Claude Code sessions, typed thoughts — all into one bank. Slack is built and off by default, as you asked.	PROVEN
5	An interview that digs, then stops	Asks until there is a scene, a detail and a realisation. Budgets its questions. Parks a moment rather than padding it. Never writes.	PROVEN
6	An idea bank you own and can edit	28 tables. Nothing can be deleted — only killed and labelled. Moments re-open when you add to them. Full-text search across everything. Exportable in one file.	PROVEN
7	Choosing what gets written next	Five-factor scoring, pillar balance, and a retelling check against everything published. Runs short and says why rather than dropping the bar.	PROVEN
8	A reference library you control	Fourteen sections in plain prose, edited in a browser or by one message. Versioned, reversible, read at drafting and at checking. Seven still await your input.	PROVEN
9	Drafting, and a gate that rejects	Eight independent checks, each able to reject alone. 130 checks have run and 48 rejected. It demonstrably rejects; it has never yet passed a draft.	UNPROVEN
10	Turning an image into your own	Rebuilds the idea as a vector drawing, so it is incapable of resembling the original rather than merely told not to. No rebuild has run yet.	UNPROVEN
11	A calendar, and a weekly pass	Drafts arrive as drafts. You edit in place and mark ready — the only thing that authorises publishing. Your edits go back into the bank automatically.	PROVEN
12	It gets better over time	Measures what it wrote against what you approved — needing no LinkedIn. Asks one question a week. Proposes library changes with evidence; you decide.	PROVEN
13	It must not break quietly	Silence alerts, queue warnings, errors surfaced once rather than daily, and a monthly report on itself. It also tells you if it starts using a service you were not told about.	PROVEN
14	You own it, and it runs without us	Everything transfers on payment; no licence needed from us. The move into your accounts is rehearsed, not just written. The accounts have not moved yet.	OUTSTANDING

#	WHAT YOU ASKED FOR	WHAT EXISTS	STATE
15	Tools, confidentiality and cost	Cost confirmed at ~\$53/month before the build. Every table behind access rules, one sign-in address. The system now tells you itself if it uses a service not on your list.	PROVEN
16	Decisions left to us	All seven recommended with reasoning before the work: storage, the bank, orchestration, models, the calendar, the library structure, and the build order.	PROVEN
17	How we both know it is finished	Twelve component tests and a finish line, computed from the live system on demand and folded into your monthly message. None passed yet; none failed.	WAITING

READING THAT TABLE

Fifteen of seventeen are proven. Two are built but unproven — the gate and the visuals — and both wait on the same thing, which is a model good enough to do the work. One is outstanding: the accounts are still ours rather than yours, and that is the first thing to change at handover.

Everything you touch is one of three things, and each exists because the other two cannot do its job. **The split is the whole design** — once it is clear, every feature in this document explains itself.

Telegram — every decision

LIVE

Capture, the interview, approving, scheduling, verdicts, clearing a name, adding a rule. Anything answerable with a tap, wherever you already are. Nothing here needs a laptop.

The desk — every prose job

LIVE

Editing a post, writing the library, finding a moment, rolling a change back, exporting everything. Four jobs a chat box physically cannot do, because each needs a cursor.

The reader — one input

NOT INSTALLED

Runs on your own laptop, reads your Claude Code sessions, sends a short digest. Set up once, then silent. The only piece that needs software on your machine.

UNDERNEATH ALL THREE

One database and ten background services on **nine schedules** — the queue every minute, selection every four hours, publishing every five minutes, a check each morning at eight, the learning run on Monday. It mines, scores, drafts, checks and reports without anyone starting it.

Why the split is drawn there

Your specification says the weekly pass must happen where you already are, because *"a separate habit will not survive"*. So every decision is in Telegram.

It also says the idea bank must be editable *"without going through the chat interface"*. So the four jobs needing a keyboard get a screen — and nothing else does.

The rule of thumb

Answerable with a tap → Telegram.
Needs a cursor → the desk.

That is why the desk is only six screens. It is not a second copy of the bot; it is the small residue of work that genuinely needs a keyboard.

The Telegram bot

Type `/` and it lists everything it can do. Most of the time you will not need any of it — you send a voice note and it asks you a question back.

COMMAND	WHAT IT DOES
<code>/review</code>	Your weekly pass. Every draft waiting, each with buttons to schedule, rewrite or reject.
<code>/status</code>	What is waiting, in the order you can act on it. The buttons change with the numbers.
<code>/ask</code>	Starts a session. Offers anything waiting from your calls first, then falls back to questions.
<code>/candidates</code>	What your calls and working sessions turned up, waiting to be dug into.
<code>/seed</code>	A long sitting — one moment straight into the next, to fill the bank in one go.
<code>/voiceguide</code>	Records you talking, and keeps it as the source of truth for how you sound.
<code>/rule</code>	Adds a rule in one message. Applies to the very next draft — nothing to deploy.
<code>/stop</code>	Ends the session. Everything captured so far is kept.
<code>/help</code>	What is waiting, then buttons that run things rather than naming them.
YOU SEND	IT DOES
A voice note	Stores the audio, transcribes it, files it as a moment, and replies with one question about it.
A typed thought	The same, without the transcription step.
An image	Asks what you are taking from it and which moment it belongs to, then rebuilds the idea in your brand.
A reply to a draft	Treated as push-back. It rewrites, and the new version goes back through all eight checks.
Anything, mid-question	If the question is hours old it asks whether this is an answer or something new, rather than guessing and losing it.

WHY IT NEVER NAGS

Nothing waits on you. A question can sit unanswered for a week; a moment parks rather than being padded out; the weekly question stops after being asked twice. **The system is built so that ignoring it has no cost.**

The desk — six screens

SCREEN	WHAT IT IS FOR
This week	Drafts waiting, the one conversation question, a verdict box on each. Opens with four figures showing where everything stands.
Calendar	Four weeks of what is going out. Edit in place and mark ready — the only thing that authorises publishing.

SCREEN	WHAT IT IS FOR
Idea bank	Every moment ever captured, searchable across your words, the scene, the detail and what they said. Pin one, kill one, correct a mangled quote.
Reference library	The fourteen sections that decide how it writes. Edit any of them and the next draft uses it. Roll any change back.
Proposals	Changes the system wants to make, each with its evidence. You approve or reject; it never changes itself.
Export	Every table, as one file, whenever you want it. No lock-in.

Search, specifically. Built as real full-text search rather than pattern matching — it finds "costs" when you type "cost", and does not return "costume". It looks across every status, because the moment you go looking for later is usually one that has already moved on. Press `Ctrl-K` anywhere to search or jump.

The Claude Code reader

Your working sessions contain real material — the bug that turned out to be an assumption, the decision that changed halfway. The logs are on your laptop and nothing in the cloud can reach them, so this runs locally.

COMMAND	WHAT IT DOES
install	Checks everything, asks for credentials once, schedules it daily, then proves it works with a dry run.
--doctor	Checks every reason it could fail, including whether the credentials actually work against the live server.
--status	Is it scheduled, and when did it last run.
--uninstall	Stops it. The agent and everything it has sent stay.

WHAT IT ACTUALLY SENDS, AND WHY --DOCTOR EXISTS

A short digest of what you said and decided. **Never code, never client material.** Of 576 files on a typical machine, 535 are discarded before anything is read, and only the genuinely unusual sessions survive after that.

A wrong key produces **silence** — and a week that produces nothing is a legitimate pass here, so a broken setup and a quiet week look identical from outside. Checking the credentials against the real server is the only way to tell them apart.

Most of the work is not a feature. It is the machinery that keeps the features honest when nobody is watching — **and it is the majority of what was built.** This section exists because none of it appears anywhere else in this document.

A database that refuses bad states

PROVEN

28 tables, 73 access rules, 31 constraints. **No delete permission exists anywhere**, so nothing you have said can be destroyed by any code path — only killed and labelled. Every table is behind row-level access, so a query cannot return material you are not entitled to even if the code asks for it.

A queue that survives failure

PROVEN

Every piece of work is a job. A job that dies half-way becomes claimable again rather than being lost, retries back off rather than hammering, and a genuine failure is separated from a temporary one — a rate limit gets forty patient attempts, a real error gets five. This is why a dropped connection at 3am does not cost you a voice note.

The retelling check

PROVEN

Every published post is indexed by meaning, not by words, so the system can tell that a new draft is the same story told differently. The thresholds were *measured* against your material rather than guessed — and when they were inherited from elsewhere they would have blocked nothing while appearing to work.

It notices its own drift

PROVEN

The system records every outside service it actually calls and tells you the first time it uses one that is not on your list. A test also fails the build if the tool list you hold and the list the system checks itself against ever disagree. **Both exist because that drift already happened once.**

95 automated tests, and four measuring harnesses

PROVEN

The tests pin the things that would fail silently: that your own underscores and asterisks survive formatting, that an unmeasured acceptance test reports as unmeasurable rather than as zero, that every button offered maps to an action. The harnesses measure the system against your own acceptance criteria and can be re-run at any time — the scorecard in section 07 is generated, not typed.

Six documents, and a rehearsed handover

PROVEN

The specification read clause by clause, the seven build decisions with their reasoning, the running cost, an operating runbook, the sequence for moving everything into your accounts, and the running order for the recorded walkthrough. The account move is **rehearsed rather than described**: all 27 database migrations were applied to an empty database to prove the sequence works before anyone depends on it.

WHY THIS MATTERS MORE THAN IT SOUNDS

Four times during this build a control was found that *looked* like it worked and did nothing — a button that reverted a post and recorded no reason, a rollback that silently matched no rows, a skip that was

never read, a scheduler entry nobody ever created. Each was found by driving the live system rather than by reading the code. **The machinery above is what makes the next one visible.**

The order is fixed and every step can stop. **Nothing skips ahead** — in particular nothing is ever drafted from a transcript or a session log without interviewing you about it first, because a recording captures what was said and not what you made of it.

#	STEP	WHO	WHAT HAPPENS
01	Something arrives	YOU	A voice note, a typed thought, a call transcript, a Claude Code session, or a Slack thread if you turn it on. All five land in the same idea bank.
02	It asks you about it	Automatic	Until the moment has a scene, a detail and a realisation. Then it stops. If it cannot get there, the moment parks rather than being padded out.
03	It picks what to write	Automatic	Scored on five factors, balanced across your pillars, checked against everything published so a story is never retold. If the bar cannot be met it writes fewer and says why.
04	It writes a draft	Automatic	Reading only your bank entry and your library. It never browses and never researches, and it emits every factual claim with its source text beside it.
05	Eight checks	Automatic	Could anyone else have written this · does every claim trace to something you said · is a client identifiable · are all names cleared · does it sound like you · does the hook earn the next line · is it aimed at someone · does it use banned phrasing. Each can reject on its own.
06	It reaches your calendar	Automatic	As a draft, never live, with its image attached if you sent one.
07	You approve it	YOU	A tap in Telegram or an edit in the calendar. This is the only thing that authorises publishing, and the database refuses to store a published post without it.
08	It publishes	Automatic	To LinkedIn, on the date you set. Never before, and never anything you did not approve.
09	It learns	Automatic	What it wrote against what you approved, the numbers after seven days, and one question a week about whether a post started a conversation. It proposes library changes with evidence; you decide.

Not policies the code tries to follow — **constraints the database will not let it violate**, which means they hold even if the code above them is wrong.

RULE	HOW IT IS HELD
Only you could have written it	Kept in a locked section of your library. The learning loop is structurally unable to propose a change to it — a database rule rejects the attempt.
Nothing invented	Every claim carries its source text and is verified mechanically, before any model gives an opinion. Missing detail stays missing, and the draft works without it.
Nothing publishes without you	Four constraints make an unauthorised published post impossible to store. Tested by trying: the database refuses all three ways of doing it.

AND ONE THING THAT IS NEVER TUNABLE

The learning loop can change how it writes — hooks, structure, pillars, timing. It can never loosen the two rules above, whatever the engagement numbers say. **A system optimising freely against LinkedIn engagement finds its way to engagement bait**, and that is the one outcome that would make the whole thing worthless.

Twelve component tests decide whether this is finished. **None has been passed and none has been failed** — every one is waiting on something specific, and this is that list.

#	TEST	WAITING ON	WHERE IT STANDS
01	Raw capture	You	1 of 20 voice notes sent. The mechanism is proven; the volume is not there.
02	Prompted session	You	3 of 5 sessions run.
03	Call transcripts	You, then 2 weeks	Needs a call recorder chosen, then two weeks of calls running through.
04	Claude Code	One install	The reader has never run. One command starts a two-week clock.
05	Interview	You	1 of 2 answered moments reached depth. Five more were asked and never answered — which measures nothing about the interview.
06	Fabrication	Claude Code	Needs 10 drafts through the gate for you to audit. There are none.
07	Names	Claude Code	Same. Four client names are on file and correctly held back.
08	The gate	Claude Code	Run for real: 6 of 80 checks completed before the free model hit its limit. Not a failure — it never got to read them.
09	Selection	4 weeks	46 runs so far. Needs four weeks of elapsed running.
10	Visuals	Claude Code	Needs a model that can see pictures. Your image is stored and queued.
11	Calendar	LinkedIn	Zero exceptions, but the four-week window has not begun. The guarantee itself is proven.
12	Learning loop	LinkedIn, 1 month	The mechanism runs. Nothing has published for it to learn from.
17a	The finish line	All of the above	Five of the last six drafts needing only light editing. No draft has been approved yet, so there is nothing to measure — reported as unmeasurable rather than as zero.

THE ONE THING THAT UNBLOCKS THE MOST

A model good enough to write and to judge. The build has been running on free tiers, which allow 8,000 words a minute where a single quality check needs roughly 16,000. That one fact blocks four component tests and the finish line itself.

The intended route is Claude Code rather than buying API credits — you already have it, and there is already a small agent running on your machine on a schedule that could carry the drafting and the checks. *This is not built yet.* It is the next substantial piece of work, and it is what those four rows are waiting on.

Worth being precise about what this is *not*: the gate is not broken. Reading its rejections shows it correctly catching genuine invention — phrases the free model added that you never said. **The safety mechanism works; the writer beneath it is the cheap one.**

Seven of the fourteen library sections are waiting on you, and they are the ones that cannot be guessed without putting words in your mouth.

The voice guide

8.1, 8.2

BLOCKING

A recording of you talking at length, in your own words — **never your old posts**, which you have said have drifted. The whole quality argument rests on this and it has no date against it. The system has already written a first draft from what it has, so you are reacting to something rather than facing a blank page. It needs about 400 words of you speaking first.

Pillars, audience, gate rules, visual brand, reference posts

clause 8

WAITING

Each section explains what belongs in it and why we did not draft it for you. Editable in a browser, or from Telegram with a single message — a rule can be added in under a minute, because that is how it will actually get maintained.

Connect LinkedIn

11, 12.1

WAITING

Posting is self-serve and needs no approval. Post analytics sits behind a review that takes time, so it is worth starting early. This begins the four-week clocks on two component tests.

Answer what is already asked, and keep sending

4, 5

WAITING

Five moments are sitting asked and unanswered. Beyond those: nineteen more voice notes, two more sessions, and a decision on which call recorder you use.

THE HONEST CONSTRAINT

The critical path is calendar time, not code. Five of the twelve tests need two to four weeks of live running, and those clocks have not started. No amount of building shortens them. The most valuable thing anyone can do this week is start one.

Two pieces of work are already scoped. Neither is in your specification, and neither is built — they are here so nothing arrives as a surprise.

Drafting and checking through Claude Code

PLANNED

Instead of buying model credits, the drafting and the eight checks would run through the Claude Code subscription you already have, driven by the same local agent that already reads your sessions on a schedule. **This is what unblocks four of the twelve acceptance tests and the finish line**, and it is the next substantial piece of work.

A connector for the system itself

PLANNED

A small server that lets Claude Code — or any similar tool — read the idea bank, check where things stand and run the system directly, without opening the app or the database. Useful for you and for whoever maintains this after handover, and the natural companion to the item above.

Build Specification v2.0 · 20 August 2026 · prepared by Josh Fryszer, Slingshot GTM, for Thought Pilot. This report covers work to 31 August 2026. Every figure was read from the live system with test fixtures excluded, and is reproducible on demand.